

Universidad Autónoma de San Luis Potosí

FACULTAD DE INGENIERÍA

CARRERA:

Ingeniería en Sistemas Inteligentes

ASIGNATURA:

Lenguajes de programación

ELABORADO POR:

369664 Antonio Jiménez José Emilio
388531 Alvarez Puente Alan Antonio

DOCENTE:

CICLO ESCOLAR:

2025 - 2026

San Luis Potosí, S.L.P.

16 de mayo de 2026

Índice

1. Estructuras de Datos y Objetos	2
Entidad	2
Atributos	3
CDiccionario	4
2. Funciones	6
Menu Entidades	6
Captura Entidad	6
Alta Entidad	6
Busca Entidad	6
Get Cabecera Entidades	6
Lee Entidad	6
Escribe Cabecera Entidades	7
Escribe Entidad	7
Inserta Entidad	7
Consulta Entidades	7
Baja Entidad	7
Elimina Entidad	7
Reescribe Entidad	7
Modifica Entidad	7
3. Diagramas	8
4. Archivo	9
5. Clave primaria	10
6. Conclusiones	11
Referencias	12

1 Estructuras de Datos y Objetos

Descripción de la Estructura Entidad:

La estructura es el molde inicial de nuestro Diccionario en formato de base de datos de acceso aleatorio. Se establece un alias `cadena` para asegurar que cada registro tenga el mismo tamaño.

- **cadena nombre:** Identificador único de cada registro de entidad (máximo 30 caracteres).
- **long atr:** Puntero tipo `long` que conecta la entidad con su lista de atributos.
- **long data:** Dirección física en el archivo donde se almacenan los registros de datos reales.
- **long sig:** Enlace a la siguiente entidad, permitiendo una estructura de lista ligada.

```
1 #ifndef ENTIDAD_H_INCLUDED
2 #define ENTIDAD_H_INCLUDED
3 #include <iostream>
4 #include <cstdio>
5
6 typedef char cadena[30];
7 struct Entidad
8 {
9     cadena nombre;
10    long atr;
11    long data;
12    long sig;
13 };
14 #endif
```

Estructura 1: Estructura de Datos de Entidad

Descripción de la Estructura Atributo:

Esta estructura es el molde para las columnas de nuestro Diccionario. Al igual que en la entidad, se usa el alias `cadena` para mantener el tamaño fijo en el archivo binario y evitar que los registros se desfasen. En esta estructura observamos las siguientes declaraciones:

- **cadena nombre:** Identificador del atributo dentro de la entidad.
- **unsigned int tipo:** Código numérico que define el tipo de dato (ej. entero, flotante, cadena).
- **unsigned int tamano:** Longitud en bytes que ocupará el valor del atributo en el registro.
- **char iskp:** Bandera (flag) que indica si el atributo funciona como llave primaria (Primary Key).
- **long sig:** Puntero lógico (offset) que enlaza con el siguiente atributo de la entidad.
- **cadena descripcion:** Información adicional o etiqueta descriptiva del campo.
- **char null:** Indicador booleano que define si el campo permite valores nulos.

```
1  #ifndef ATRIBUTO_H
2  #define ATRIBUTO_H
3
4  typedef char cadena [30];
5
6  typedef struct{
7      cadena nombre;
8      unsigned int tipo;
9      unsigned int tamano;
10     char iskp;
11     long sig;
12     cadena descripcion;
13     char null;
14 }Atributo;
15
16 #endif
```

Estructura 2: Estructura de Datos de Atributos

Descripción del Objeto CDiccionario:

La clase CDiccionario funciona como el administrador principal del sistema. En su parte privada, contiene las variables necesarias para controlar el flujo del archivo binario y saber exactamente en qué parte del disco estamos parados mientras el usuario navega. En esta sección observamos las siguientes declaraciones:

- **FILE *arch:** Es el puntero al archivo físico donde se almacenan todas las entidades, atributos y datos.
- **Entidad activa:** Representa la entidad que el usuario está manipulando o consultando en ese momento.
- **long dir:** Guarda la dirección física (offset) actual dentro del archivo para las operaciones de lectura/escritura.
- **int nAtributos:** Es un contador que lleva el registro de cuántos atributos tiene la entidad seleccionada.
- **long tambloque:** Almacena el tamaño del bloque activo para gestionar correctamente el espacio en el archivo.
- **char nombreArchivo[50]:** Es el arreglo donde se guarda el nombre del archivo que el usuario decide abrir o crear.

```
1 #ifndef CDICCIONARIO_H_INCLUDED
2 #define CDICCIONARIO_H_INCLUDED
3
4 #include <cstdio>
5 #include "Atributo.h"
6 #include "Entidad.h"
7
8 class CDiccionario
9 {
10 private:
11     FILE *arch;
12     Entidad activa;
13     long dir;
14     int nAtributos;
15     long tambloque;
16
17     char nombreArchivo[50];
18 public:
19     //constructor
20     CDiccionario();
21
22     //Entidades
23     void Menu_Entidades();
24     Entidad CapturaEntidad();
25     void altaEntidad();
26     long buscaEntidad(char nombre[30]);
27     long getcabeceraEntidades();
```

```
28     Entidad leeEntidad(long dir);
29     void EscribeCabEntidades(long cab);
30     long escribeEntidad(Entidad ant);
31     void insertaEntidad(Entidad nvo, long dir);
32     void ConsultaEntidades();
33     void BajaEntidad();
34     long EliminaEntidad(char nombre[30]);
35     void reescribeEntidad(long dir, Entidad ant);
36     void modificaEntidad();
37
38     //Atributos
39     void Menu_Atributos();
40     void consultarAtributo();
41     long eliminaAtributo(char cad[30]);
42     void modificaAtributo();
43     void AltaAtributo();
44     Atributo CapturaAt();
45     void insertaAtributo(Atributo nvo, long dir);
46     long BuscarAtributo(char *atr);
47     Atributo leeAtributo(long dir);
48     long EscribeAtributo(Atributo ant);
49     void reescribeAtributo(long dir, Atributo atr);
50     void BajaAtributo();
51
52     //Datos
53     void Menu_Datos();
54
55     //Principal
56     void menu();
57     void Nuevo();
58     void abrir();
59
60 };
61
62 #endif
```

Estructura 3: Estructura de Datos CDiccionario

2 Funciones

Entidades

Menu Entidades

La función `menu_Entidades()` muestra un menu interactivo por consola para administrar las funciones relacionadas a Entidades. Funciona por medio de un ciclo `do-while` y un `switch-case` que lee la opción del usuario, para ejecutar acciones como dar de alta, consultar, eliminar o modificar entidades, por consiguiente permitir la navegacion hacia la edicion de sus atributos y datos.

Captura Entidad

La función `CapturaEntidad()` es la plantilla principal para crear registros, se usa una variables de tipo `Entidad`, primero se le pide al usuario el nombre de la entidad, despues se inicializa las variables `nueva.atr` conexion con la lista de atributos", `nueva.data` "la direcciónreal de donde se encuentran los datos de los bloques, y `nueva.sig` que es la dirección a la siguiente entidad, con esto se crea un registro nuevo con sus listas vacias.

Alta Entidad

Esta función es la encargada de verificar que todo este en condiciones para que se cree una entidad primero manda a llamar `CapturaEntidad` para crear un registro temporal, seguido de esto viene la verificacion con la funcion `buscaEntidad` para verificar que no haya una funcion con ese mismo nombre, en caso que no exista usa la funcion `escribeEntidad(nueva)` y guarda la dirección en un `long dir` para finalmente usar la función `insertaEntidad` para escribir en el archivo.

Busca Entidad

La función `long CDiccionario::buscaEntidad(char nombre[])` verifica si se encuentra la entidad, en caso de que si la encuentre dentro del recorrido `while(cab != -1)` regresa la dirección en donde se encuentra, en caso contrario regresa un valor de -1, dentro del ciclo hacer un string compare de la posicion donde se encuentra y el nombre que se le haya pasado.

Get Cabecera Entidades

Esta función es la encargada de obtener la direccion dentro del archivo de donde inicia nuestra lista de entidades, empieza usando un `long dir`, para despues con `fseek` dirigirse al inicio de nuestro archivo, con un `fread` extraemos la direccion y la guardamos en la variable `long dir`, sabemos que esa es la cabecera ya que todo el programa esta construido que en el inicio del archivo se encuentra nuestra cabecera de la lista de entidades.

Lee Entidad

Escribe Cabecera Entidades

Escribe Entidad

Inserta Entidad

Consulta Entidades

Baja Entidad

Elimina Entidad

Reescribe Entidad

Modifica Entidad

3 Diagramas

4 Archivo

5 Clave primaria

6 Conclusiones

Referencias
